3D WEBSOFT INC. FOUNDATION

FUTURE TECH SCHOOL PROGRAM

Faculty Name :

Course Duration : 5 Months

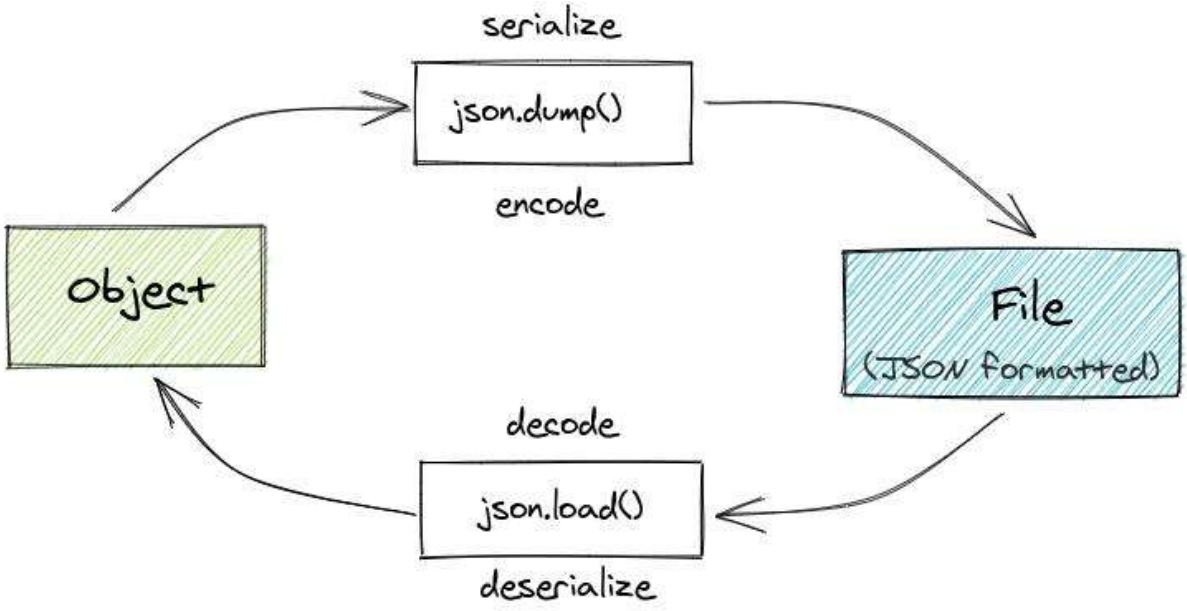
Classes : Offline & Online

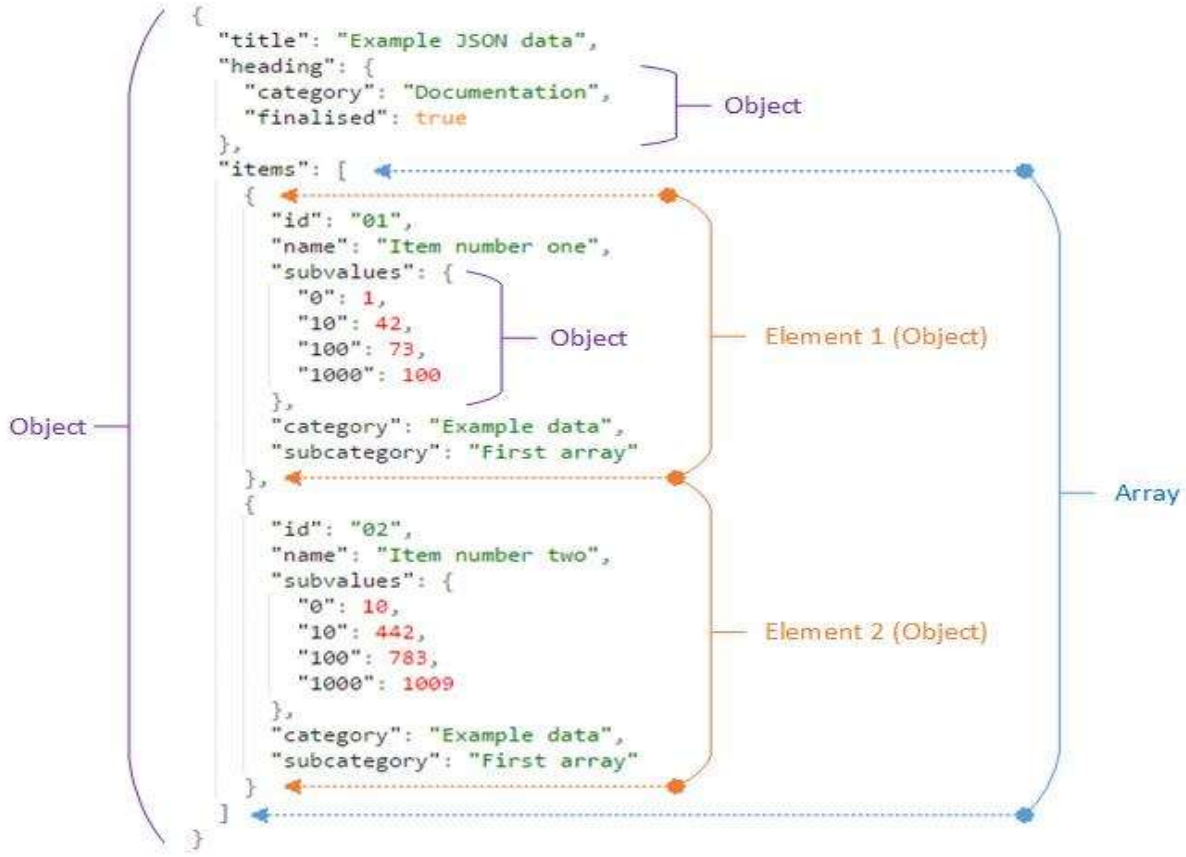
Daily Teaching : 2 Hours

Total Classes : 100

Level 4 – AI Professional Foundation(11th–12th Standard)

Class 16 – JSON Data Handling






```
{
  "students" : [
    {
      "firstName" : "Jane",
      "lastName" : "Doe",
      "major" : "CENT"
    },
    {
      "firstName" : "John",
      "lastName" : "Doe",
      "major" : "PHYS"
    },
    {
      "firstName" : "Bill",
      "lastName" : "Gates",
      "major" : "ICS"
    }
  ]
}
```

1. Learning Objectives

By the end of this lesson, students will be able to:

- Understand what JSON is and why it is used.
- Identify JSON **objects, arrays, keys and values**.
- Create JSON data in Python.
- Convert Python objects into JSON.
- Convert JSON data back into Python objects.
- Read and write .json files.
- Add, search, update and delete student records.
- Build a simple **JSON-based Student Management Application**.

2. What is JSON?

JSON stands for:

JavaScript Object Notation

JSON is a lightweight format used to **store and exchange data**.

Although JSON originated from JavaScript, it is widely used with Python, Java, C++, JavaScript, web applications, APIs and databases.

Simple JSON example

```
{
   "name": "Rahul",
   "age": 17,
   "course": "Python"
}
```

3DWEBSOFT FOUNDATION

Here:

Part	Meaning
"name"	Key
"Rahul"	Value
"age"	Key
17	Value
"course"	Key
"Python"	Value

Easy memory trick

JSON = Data arranged in a structured format.

3. Why Do We Need JSON?

Suppose an application needs to store student information.

Without structured data:

Rahul,17,Python,85

Priya,16,Python,92

Arjun,17,Python,78

It can become difficult to understand and process.

Using JSON:

```
[
  {
    "name": "Rahul",
```

3DWEBSOFT FOUNDATION

```
    "age": 17,
    "course": "Python",
    "marks": 85
  },
  {
    "name": "Priya",
    "age": 16,
    "course": "Python",
    "marks": 92
  }
]
```

The structure is much clearer.

4. Important JSON Data Types

JSON supports several basic data types.

JSON Type	Example
String	"Rahul"
Number	17
Boolean	true
Null	null
Object	{ }
Array	[ ]

3DWEBSOFT FOUNDATION

Example

```
{
   "name": "Rahul",
   "age": 17,
   "active": true,
   "phone": null
}
```

5. JSON Object

A JSON object is written using:

{ }

It contains **key-value pairs**.

Example:

```
{
   "name": "Anjali",
   "age": 16,
   "marks": 90
}
```

Think of it like a Python dictionary.

Python

```
student = {
   "name": "Anjali",
   "age": 16,
```

3DWEBSOFT FOUNDATION

```
    "marks": 90
}
```

JSON

```
{
    "name": "Anjali",
    "age": 16,
    "marks": 90
}
```

6. JSON Array

A JSON array is written using:

[ ]

It stores multiple values.

Example:

```
[
    "Python",
    "Java",
    "C++"
]
```

A list of students can be represented as:

```
[
    {
        "name": "Rahul",
```

3DWEBSOFT FOUNDATION

```
    "marks": 85
  },
  {
    "name": "Priya",
    "marks": 92
  }
]
```

7. JSON vs Python Dictionary

JSON and Python dictionaries look similar, but they are not exactly the same thing.

Python	JSON
dict	Object
list	Array
True	true
False	false
None	null
Usually used inside Python	Used for data storage/exchange

Important

Python:

True

False

3DWEBSOFT FOUNDATION

None

JSON:

true

false

null

8. Python JSON Module

Python provides a built-in module called:

json

We import it using:

import json

No separate installation is required.

9. Python Object → JSON

The process of converting Python data into JSON is called **serialization**.

Important functions

json.dumps()

json.dump()

10. json.dumps()

dumps() converts a Python object into a JSON-formatted string.

Example

3DWEBSOFT FOUNDATION

```
import json

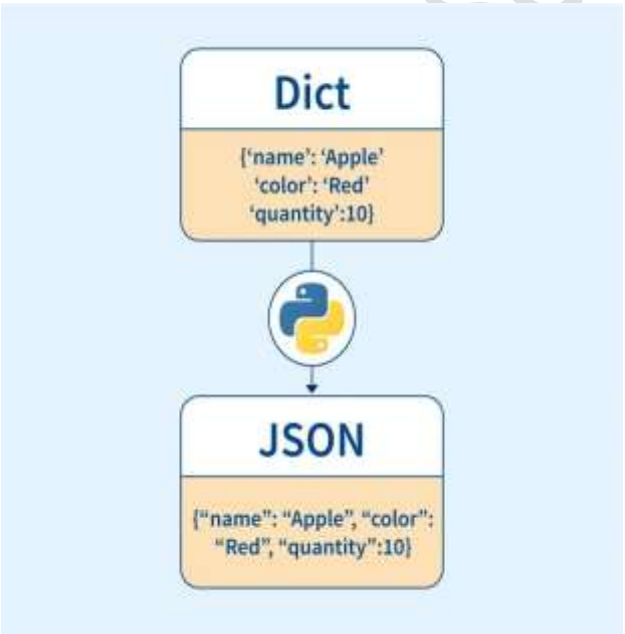
student = {
    "name": "Rahul",
    "age": 17,
    "marks": 85
}

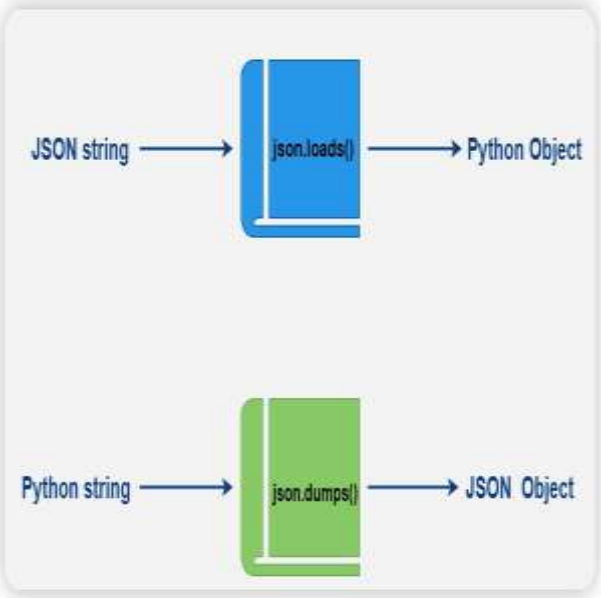
data = json.dumps(student)
print(data)
```

Output

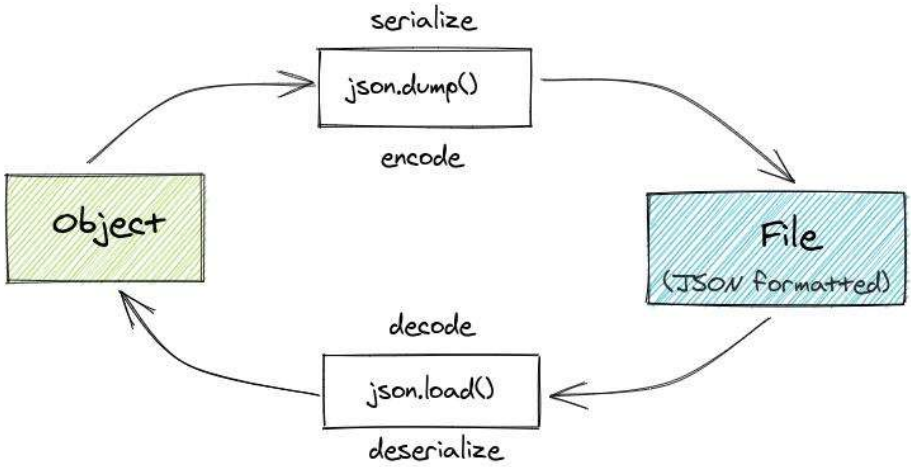
```
{"name": "Rahul", "age": 17, "marks": 85}
```

Visual Flow





SOFT FOUNDATION



Python Dictionary

↓

json.dumps()

↓

JSON String

11. Pretty JSON Output

We can make JSON easier to read using indent.

```
import json


student = {
   "name": "Rahul",
   "age": 17,
   "course": "Python",
   "marks": 85
}
```

3DWEBSOFT FOUNDATION

```
data = json.dumps(student, indent=4)

print(data)
```

Output

```
{
   "name": "Rahul",
   "age": 17,
   "course": "Python",
   "marks": 85
}
```

Why use indent?

It makes JSON:

- Easier to read
- Easier to debug
- Better organized

12. Python JSON → Python Object

The reverse process is called **deserialization**.

We can use:

json.loads()

Example

```
import json


data = '{"name": "Rahul", "age": 17}'
```

3DWEBSOFT FOUNDATION

```
student = json.loads(data)

print(student)

print(student["name"])

print(student["age"])
```

Output

```
{'name': 'Rahul', 'age': 17}

Rahul

17
```

Flow

JSON String

↓

json.loads()

↓

Python Dictionary

13. dump() vs dumps()

This is an important exam concept.

Function	Purpose
json.dump()	Write JSON to a file
json.dumps()	Convert Python object to JSON string
json.load()	Read JSON from a file
json.loads()	Convert JSON string to Python object

9DWEBSOFT FOUNDATION

Memory Trick

S = String

dumps → string

loads → string

Without s:

dump → file

load → file

14. Writing JSON to a File

We can store JSON data in a file such as:

students.json

Example

```
import json


student = {
   "name": "Rahul",
   "age": 17,
   "course": "Python",
   "marks": 85
}
with open("students.json", "w") as file:
   json.dump(student, file, indent=4)
print("Data saved successfully!")
```

3DWEBSOFT FOUNDATION

The file will contain:

```
{
   "name": "Rahul",
   "age": 17,
   "course": "Python",
   "marks": 85
}
```

15. Reading JSON from a File

Use:

json.load()

Example

```
import json

with open("students.json", "r") as file:
   student = json.load(file)
print(student)
print("Name:", student["name"])
print("Marks:", student["marks"])
```

Output

```
{'name': 'Rahul', 'age': 17, 'course': 'Python', 'marks': 85}
Name: Rahul
Marks: 85
```

SUWEBSOFT FOUNDATION

16. Storing Multiple Students

A real application needs multiple records.

```
[
  {
    "roll": 101,
    "name": "Rahul",
    "marks": 85
  },
  {
    "roll": 102,
    "name": "Priya",
    "marks": 92
  },
  {
    "roll": 103,
    "name": "Arjun",
    "marks": 78
  }
]
```

Notice:

- [ ] → multiple students
- { } → one student
- "roll" → key

3DWEBSOFT FOUNDATION

- 101 → value

Practical Session

17. Practical 1 – Create Student JSON

```
import json


students = [
  {
    "roll": 101,
    "name": "Rahul",
    "marks": 85
  },
  {
    "roll": 102,
    "name": "Priya",
    "marks": 92
  },
  {
    "roll": 103,
    "name": "Arjun",
    "marks": 78
```

3DWEBSOFT FOUNDATION

```
    }
]

print(json.dumps(students, indent=4))
```

Expected Output

```
[
    {
        "roll": 101,
        "name": "Rahul",
        "marks": 85
    },
    {
        "roll": 102,
        "name": "Priya",
        "marks": 92
    },
    {
        "roll": 103,
        "name": "Arjun",
        "marks": 78
    }
]
```

3DWEBSOFT FOUNDATION

18. Practical 2 – Save Multiple Students

```
import json


students = [
    {
        "roll": 101,
        "name": "Rahul",
        "marks": 85
    },
    {
        "roll": 102,
        "name": "Priya",
        "marks": 92
    },
    {
        "roll": 103,
        "name": "Arjun",
        "marks": 78
    }
]


with open("students.json", "w") as file:
    json.dump(students, file, indent=4)
```

3DWEBSOFT FOUNDATION

```
print("Student data saved!")
```

19. Practical 3 – Read All Students

```
import json

with open("students.json", "r") as file:
    students = json.load(file)

for student in students:
    print("Roll:", student["roll"])
    print("Name:", student["name"])
    print("Marks:", student["marks"])
    print("----------------")
```

Output

```
Roll: 101
Name: Rahul
Marks: 85
----------------
Roll: 102
Name: Priya
Marks: 92
----------------
```

SUWEBSOFT FOUNDATION

Roll: 103

Name: Arjun

Marks: 78

---

20. Practical 4 – Search Student

```
import json

with open("students.json", "r") as file:
    students = json.load(file)

roll = int(input("Enter roll number: "))

found = False

for student in students:
    if student["roll"] == roll:
        print("Student Found!")
        print("Name:", student["name"])
        print("Marks:", student["marks"])
        found = True
        break
```

3DWEBSOFT FOUNDATION

```
if not found:
    print("Student not found.")
```

Example Output

```
Enter roll number: 102
Student Found!
Name: Priya
Marks: 92
```

21. Practical 5 – Add a Student

To add a new student:

1. Read existing JSON.
2. Create a new dictionary.
3. Add it to the list.
4. Write the list back to the file.

```
import json

with open("students.json", "r") as file:
    students = json.load(file)

roll = int(input("Enter roll number: "))
name = input("Enter name: ")
marks = int(input("Enter marks: "))
```

3DWEBSOFT FOUNDATION

```
new_student = {
    "roll": roll,
    "name": name,
    "marks": marks
}
students.append(new_student)
with open("students.json", "w") as file:
    json.dump(students, file, indent=4)
print("Student added successfully!")
```

22. Practical 6 – Calculate Average Marks

```
import json

with open("students.json", "r") as file:
    students = json.load(file)

total = 0

for student in students:
    total += student["marks"]

average = total / len(students)
```

3DWEBSOFT FOUNDATION

```
print("Average Marks:", average)
```

Example Output

```
Average Marks: 85.0
```

23. Practical 7 – Find Top Student

```
import json

with open("students.json", "r") as file:
    students = json.load(file)

top_student = max(students, key=lambda x: x["marks"])

print("Top Student:", top_student["name"])
print("Marks:", top_student["marks"])
```

Output

```
Top Student: Priya
Marks: 92
```

This program also connects the previous lesson's **lambda functions** with JSON data handling.

24. JSON-Based Student Management Application

Now students can combine everything learned in this lesson.

Application Features

SUWEBSOFT FOUNDATION

STUDENT APPLICATION

|

| | | | |

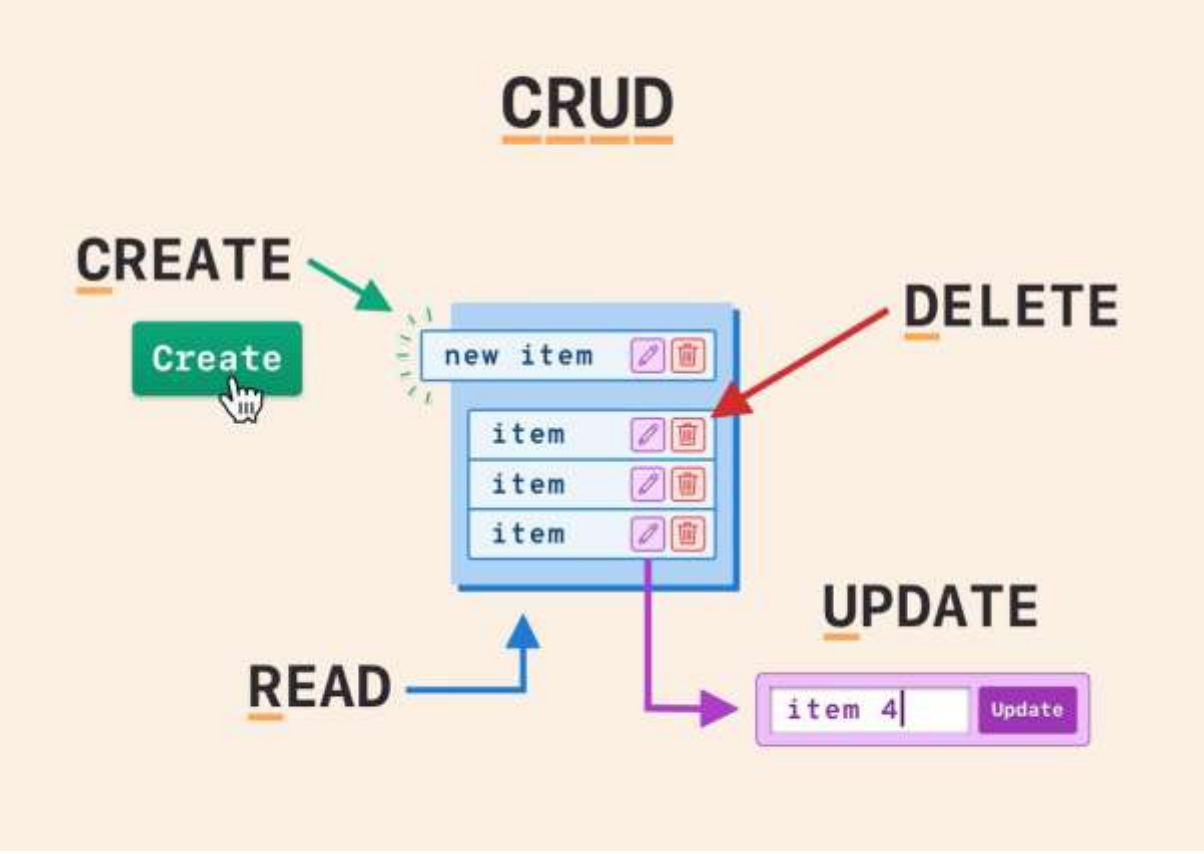
Add View Search Update Delete

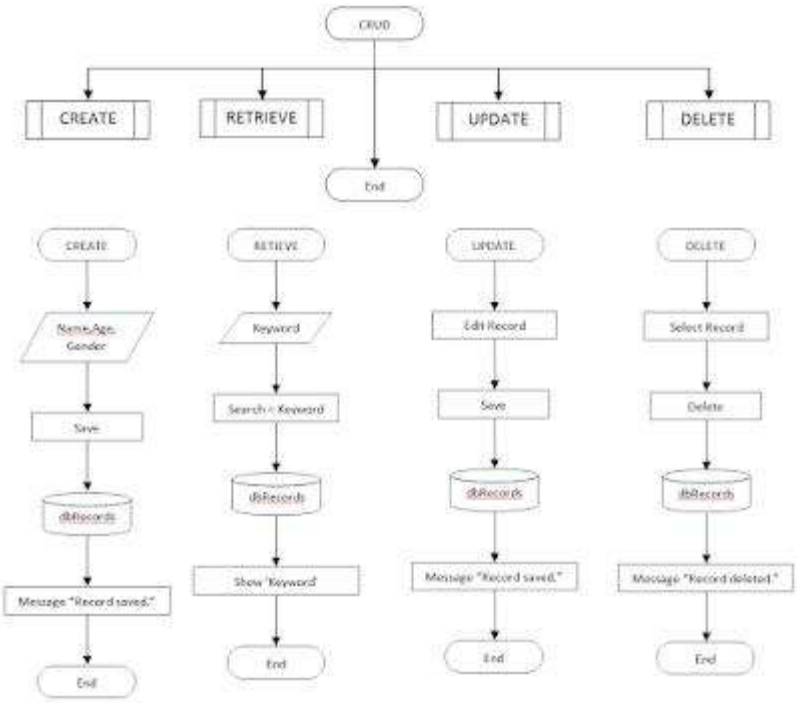
|

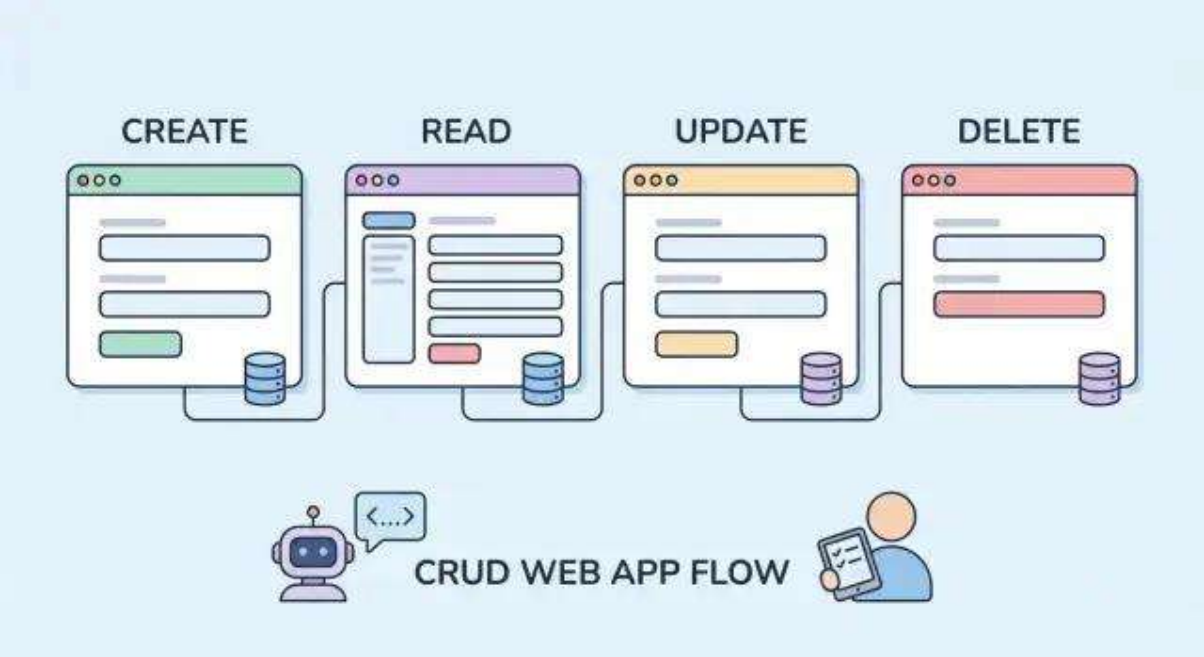
JSON

|

students.json







25. Complete Mini Application

student_json_app.py

```
import json
```

```
FILE_NAME = "students.json"



def load_students():
    try:
        with open(FILE_NAME, "r") as file:
            return json.load(file)
    except FileNotFoundError:
        return []
def save_students(students):
    with open(FILE_NAME, "w") as file:
        json.dump(students, file, indent=4)
def add_student():
    students = load_students()
    roll = int(input("Enter roll number: "))
    name = input("Enter student name: ")
    marks = float(input("Enter marks: "))
    student = {
        "roll": roll,
        "name": name,
        "marks": marks
    }
    students.append(student)
```

3DWEBSOFT FOUNDATION

```
    save_students(students)
    print("Student added successfully!")
def view_students():
    students = load_students()
    if not students:
        print("No student records found.")
        return
    for student in students:
        print("----------------------")
        print("Roll:", student["roll"])
        print("Name:", student["name"])
        print("Marks:", student["marks"])
def search_student():
    students = load_students()
    roll = int(input("Enter roll number to search: "))
    for student in students:
        if student["roll"] == roll:
            print("Student Found!")
            print("Name:", student["name"])
            print("Marks:", student["marks"])
            return
    print("Student not found.")
while True:
```

3DWEBSOFT FOUNDATION

```
print("\n===== STUDENT MANAGEMENT =====")
print("1. Add Student")
print("2. View Students")
print("3. Search Student")
print("4. Exit")
choice = input("Enter your choice: ")
if choice == "1":
    add_student()
elif choice == "2":
    view_students()
elif choice == "3":
    search_student()
elif choice == "4":
    print("Application closed.")
    break
else:
    print("Invalid choice!")
```

3DWEBSOFT FOUNDATION

26. Sample Application Run

```
===== STUDENT MANAGEMENT =====
1. Add Student
2. View Students
3. Search Student
```

4. Exit

Enter your choice: 1

Enter roll number: 101

Enter student name: Rahul

Enter marks: 85

Student added successfully!

Then:

Enter your choice: 2

Roll: 101

Name: Rahul

Marks: 85.0

27. Improving the Application

Ask students to extend the application by adding:

Option 5 – Update Student

Enter roll number: 101

Enter new marks: 90

Student updated successfully!

Option 6 – Delete Student

Enter roll number: 101

Student deleted successfully!

Option 7 – Statistics

SOWEBSOFT FOUNDATION

Display:

Total Students: 10

Average Marks: 78.5

Highest Marks: 95

Lowest Marks: 52

28. JSON Application Architecture

Students should understand how different components work together.

USER

↓

Python Application

↓

Functions / Logic

↓

JSON Module

↓

students.json

↓

Stored Student Data

This is the basic idea behind many real applications that exchange or store structured data.

3DWEBSOFT FOUNDATION

29. JSON and APIs

JSON is extremely important in modern programming because many **web APIs** use JSON to exchange data.

Example response:

```
{
   "id": 101,
   "name": "Rahul",
   "course": "Python",
   "status": "active"
}
```

A Python program can receive this data and process it.

Typical flow

Python Application

↓

Request

↓

API

↓

JSON Response

↓

Python Application

↓

Display Information

3DWEBSOFT FOUNDATION

30. Common JSON Errors

Error 1 – Using single quotes in JSON

Incorrect JSON:

```
{'name': 'Rahul'}
```

Standard JSON uses:

```
{
   "name": "Rahul"
}
```

Error 2 – Missing comma

Incorrect:

```
{
   "name": "Rahul"
   "age": 17
}
```

Correct:

```
{
   "name": "Rahul",
   "age": 17
}
```

Error 3 – Wrong Boolean

Incorrect:

```
{
   "active": True
```

3DWEBSOFT FOUNDATION

}

Correct:

{

"active": true

}

Error 4 – Wrong function

Remember:

json.dump() # file

json.dumps() # string

json.load() # file

json.loads() # string

31. Classroom Activity

Activity: Build Your Own Student JSON

Ask students to create a JSON record containing:

- Roll number
- Name
- Age
- Course
- Marks
- Email
- City

3DWEBSOFT FOUNDATION

Example:

```
{
   "roll": 101,
   "name": "Aisha",
   "age": 17,
   "course": "Python",
   "marks": 88,
   "email": "aisha@example.com",
   "city": "Hyderabad"
}
```

Then convert it into a Python dictionary using:

json.loads()

32. Practical Exercises

Exercise 1 – Employee JSON

Create:

```
{
   "id": 1001,
   "name": "John",
   "department": "IT",
   "salary": 35000
}
```

Save it into employee.json.

3DWEBSOFT FOUNDATION

Exercise 2 – Product JSON

Create a list of five products containing:

Product ID

Product Name

Price

Quantity

Exercise 3 – Student Search

Create a program that searches students by:

Roll Number

Exercise 4 – Student Statistics

Calculate:

- Total students
- Average marks
- Highest marks
- Lowest marks

Exercise 5 – JSON Backup

Create a program that copies student data from one JSON file to another.

3DWEBSOFT FOUNDATION

33. Mini Project – JSON Student Database

Project Name

Student JSON Database Application

Requirements

The application should have:

==========

STUDENT JSON DATABASE

==========

1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Student Statistics
7. Exit

JSON file

students.json

Student structure

```
{
   "roll": 101,
   "name": "Rahul",
   "age": 17,
   "course": "Python",
   "marks": 85
}
```

Skills Practiced

3DWEBSOFT FOUNDATION

Students will practice:

- Dictionaries
- Lists
- Functions
- Loops
- Conditions
- File handling
- Exception handling
- JSON
- Lambda functions
- Data processing

34. Quick Revision

Concept	**Function / Syntax**
Import JSON	import json
Python → JSON string	json.dumps()
JSON string → Python	json.loads()
Python → JSON file	json.dump()
JSON file → Python	json.load()
JSON object	{ }
JSON array	[ ]

3DWEBSOFT FOUNDATION

Concept	Function / Syntax
Pretty formatting	indent=4

Remember

STRING:

dumps ↔ loads

FILE:

dump ↔ load

35. Knowledge Check

MCQ 1

What does JSON stand for?

A. Java Standard Object Notation
B. JavaScript Object Notation
C. Java System Object Network
D. JSON Standard Object Network

Answer: B

MCQ 2

Which module is used for JSON in Python?

A. csv
B. json
C. data
D. file

Answer: B

3DWEBSOFT FOUNDATION

MCQ 3

Which function converts Python data into a JSON string?

A. json.load()
B. json.loads()
C. json.dump()
D. json.dumps()

Answer: D

MCQ 4

Which function reads JSON from a file?

A. json.load()
B. json.loads()
C. json.read()
D. json.get()

Answer: A

MCQ 5

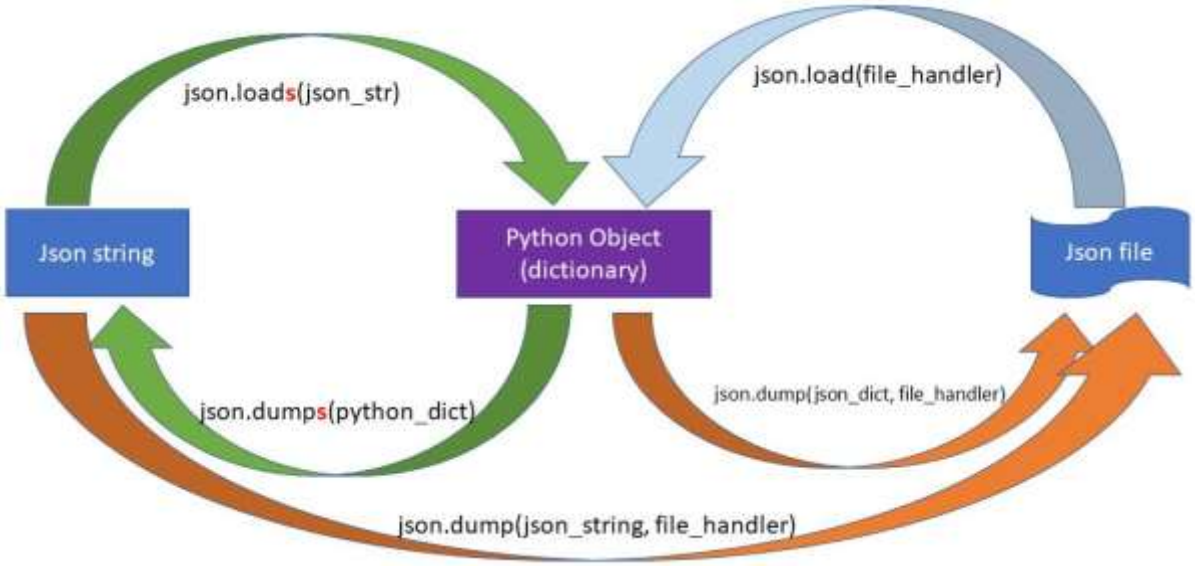
Which symbol represents a JSON array?

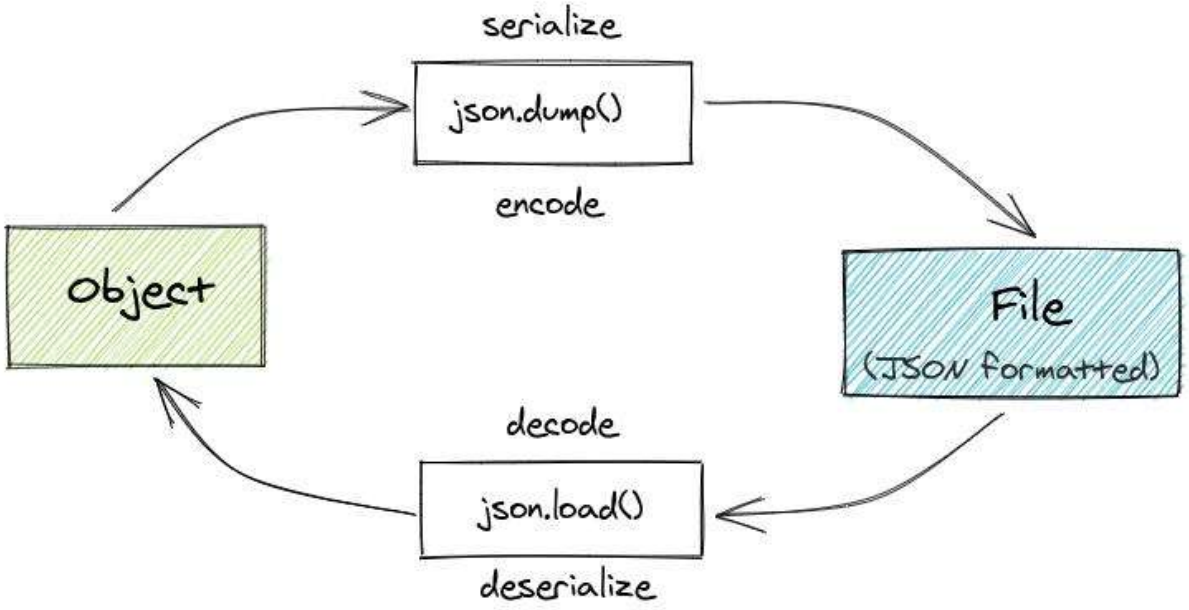
A. { }
B. ( )
C. [ ]
D. < >

Answer: C

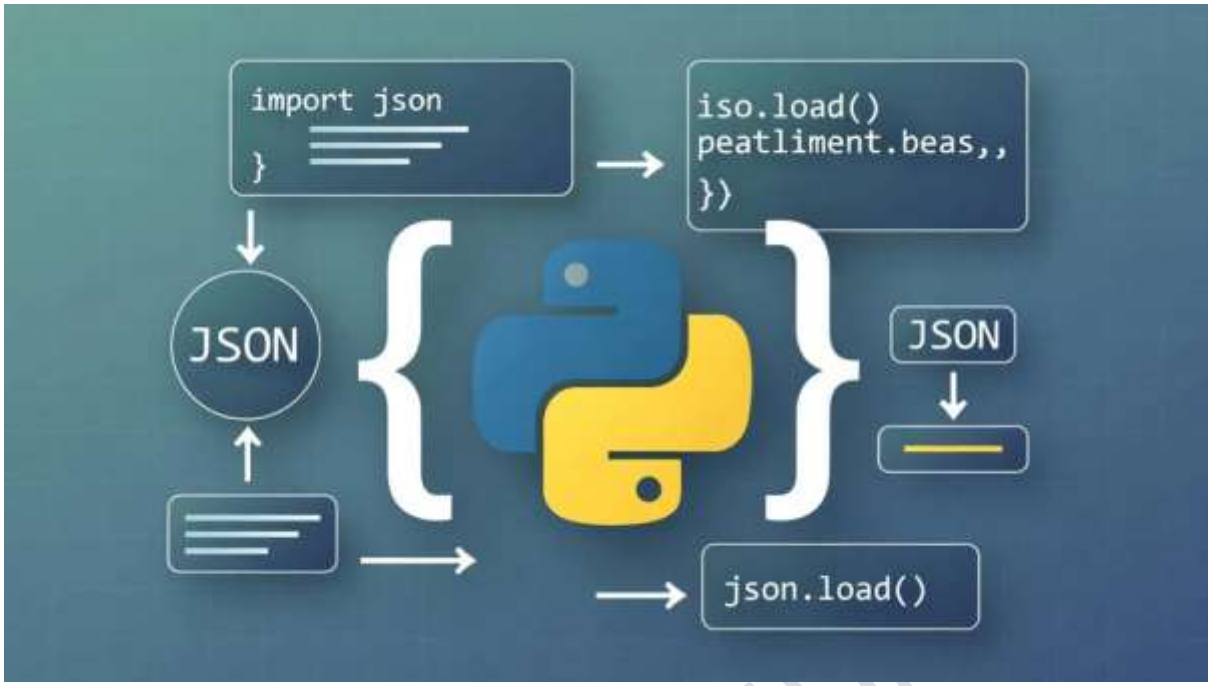
3DWEBSOFT FOUNDATION

36. Final Concept Map


json.loads(json_str)
json.load(file_handler)
Json string
Python Object
(dictionary)
Json file
json.dumps(python_dict)
json.dump(json_dict, file_handler)
json.dump(json_string, file_handler)


serialize
json.dump()
encode
object
File
(JSON formatted)
decode
json.load()
deserialize

3DWE



JSON

|

| | |

OBJECT ARRAY DATA TYPES

| | |

{ key:value } [ ] String / Number

Boolean / Null

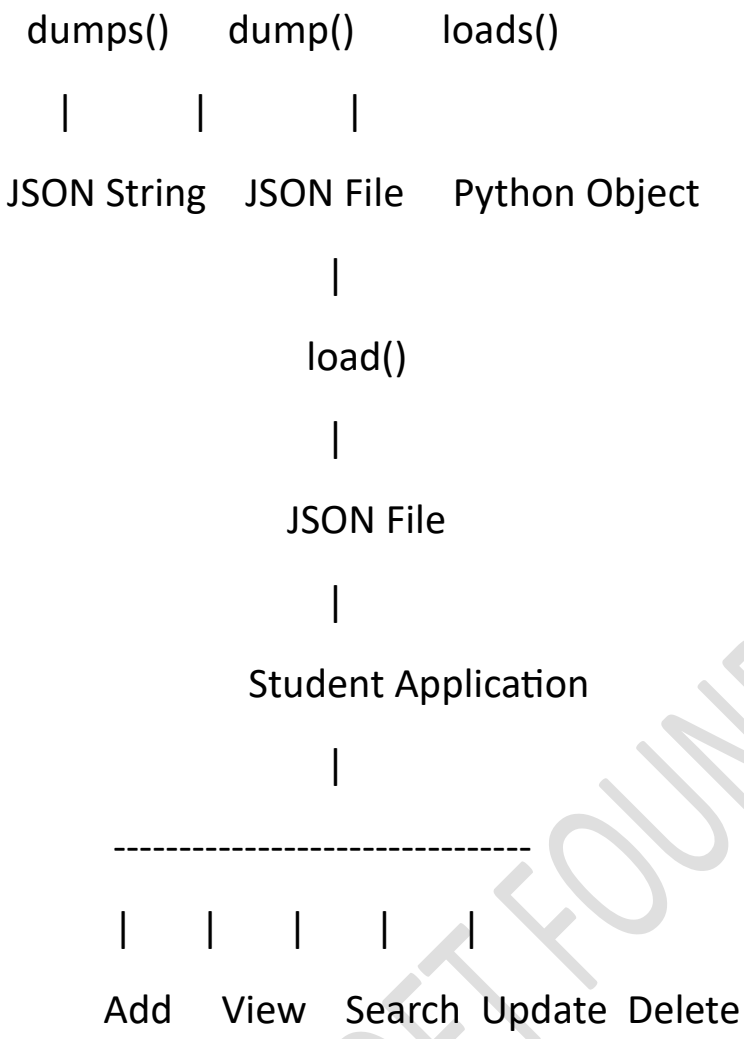
|

Python json Module

|

| | |

3DWEBSOFT FOUND



3DWEBSOFT FOUNDATION

Class 16 Takeaway

JSON is a structured way of representing data. In Python, the json module allows students to create, convert, read, write and process JSON data. By building the Student JSON Database, students connect their previous knowledge of functions, dictionaries, lists, lambda functions and file handling to a practical application.